# HELLA company 업무관리사이트 — AGENTS / clone-spec / SKILL Workflow / Backend States 통합 준비 문서

이 문서는 AI 에이전트가 HELLA company 업무관리사이트를 안전하게 이해하고, 망가뜨리지 않고, 초원자 단위로 구현할 수 있도록 만든 **실행 통제 문서**다.

문서 구성: 1. 프로젝트 헌법 및 절대 규칙 명세서 (AGENTS.md 용) 2. UI/UX 및 브랜드 1:1 치환 매핑서 (clone-spec.md 용) 3. 초원자 단위 실행 계획서 (SKILL.md Workflow 용) 4. 백엔드 연동 및 예외 상태 정의서 (Boring States 포함)

이 문서는 실제 설계 완료 상태를 전제로 하며, 추가 기획 문서가 아니라 **개발 안전장치 + 구현 지침 + 예외 처리 기준 문서**다.

---

# 0. 기준 요약

## 0-1. 프로젝트 정체성

- 프로젝트명: HELLA company 업무관리사이트
- 성격: 내부 직원 전용 통합 운영 웹앱
- 운영 구조: 관리자 데스크탑 우선 / 사원 모바일 우선 / 전체 반응형 웹앱
- 인증 방식: 사원 계정 + 비밀번호 로그인
- MVP 범위: 정산관리 제외

## 0-2. 핵심 기능 축

- 로그인 및 권한 분기
- 관리자 대시보드
- 사업분야 관리
- 메뉴얼 관리
- 일정 생성/배정/상태관리
- 사원 일정 조회
- 시작/종료/마감/사진 업로드
- 정기청소 참여 기록
- NFC 장비 반출/반입
- 미반입 장비 추적
- 감사 로그 / 다운로드 로그 / 설정

## 0-3. 역할 구조

- EMPLOYEE
- LEADER
- ADMIN
- SUPER_ADMIN

## 0-4. 시스템 철학

1. 상태 기반 설계
2. 트랜잭션 기반 처리
3. UI와 로직 완전 분리
4. 서버 기준 상태 판정
5. 모든 중요 액션 로그 필수
6. 사원은 입력 중심, 상태 확정은 서버 중심

---

# 1. 프로젝트 헌법 및 절대 규칙 명세서 (AGENTS.md)

이 장은 AI 에이전트가 절대 어기면 안 되는 프로젝트 헌법이다.

---

## 1-1. 프로젝트 정체성 선언

이 프로젝트는 **현장 서비스 회사의 통합 운영 웹앱**이다. 이 프로젝트는 단순 UI 앱이 아니다. 일정, 현장 실행, 사진 증빙, 사원 기록, NFC 장비 흐름, 로그가 서로 연결된 운영 시스템이다.

따라서 에이전트는 화면만 예쁘게 수정하는 방식으로 접근하면 안 된다. 모든 수정은 아래 기준을 따라야 한다. - 권한 - 상태 전이 - 데이터 무결성 - 로그 보존 - 관리자/사원 라우팅 분리

---

## 1-2. 하드 제약 (Critical Constraints)

### 절대 수정 금지 — 정책 / 기준 문서

아래 문서의 의미를 임의로 바꾸는 행위 금지 - docs/01_mvp_spec.md - docs/02_menu_function_specs.md - docs/03_project_roadmap.md - docs/06_permission_policy.md - docs/07_firestore_rules.md - docs/08_nfc_engine.md

### 절대 무단 변경 금지 — 보안 / 데이터

- Firestore 컬렉션 이름 변경 금지
- Firestore 핵심 필드명 무단 변경 금지
- Firestore 인덱스 무단 삭제/변경 금지
- Firestore Rules를 느슨하게 변경 금지
- Storage Rules를 공개형으로 완화 금지
- audit_logs 수정/삭제 허용 금지
- equipment_logs 클라이언트 직접 쓰기 허용 금지
- role/isActive/currentHolder/status 필드 클라이언트 자유 수정 허용 금지

### 절대 무단 변경 금지 — 상태 전이

- schedules.status를 클라이언트 직접 set하는 구조로 바꾸지 말 것
- equipments.status를 클라이언트 직접 set하는 구조로 바꾸지 말 것
- schedule_workers.workStatus를 검증 없이 임의 업데이트 가능하게 만들지 말 것
- 이미 제출된 마감 기록을 사원 권한으로 수정 가능하게 만들지 말 것

### 절대 무단 변경 금지 — 인증 / 권한

- 로그인 후 권한 분기 규칙 변경 금지
- 비활성 계정 로그인 허용 금지
- EMPLOYEE가 관리자 라우트에 접근 가능하도록 우회 금지
- ADMIN/SUPER_ADMIN만 가능한 메뉴를 공통 노출로 바꾸지 말 것

### 절대 무단 변경 금지 — NFC / 장비

- 반출/반입을 단순 UI 토글 처리로 바꾸지 말 것
- NFC 처리 시 서버 재검증 생략 금지
- 장비 상태 변경과 로그 생성을 분리 커밋하지 말 것
- 동일 장비 동시 반출 충돌 방어 제거 금지

### 절대 무단 변경 금지 — 운영 범위

- 정산/결제 기능 추가 금지
- 외부 고객용 기능 추가 금지
- 회원가입/비밀번호찾기/공개 접근 기능 임의 추가 금지

---

## 1-3. 기술 스택 및 언어 환경

### 프론트엔드

- React 18
- Vite
- TypeScript
- React Router
- Tailwind CSS
- Zustand
- TanStack Query

### 백엔드

- Firebase Auth
- Firestore
- Firebase Storage
- Firebase Functions
- Firebase Hosting

### 디바이스 / 플랫폼

- 관리자: 데스크탑 우선
- 사원: 모바일 우선
- NFC: 안드로이드 Chrome 계열 Web NFC 기준

### 파일/코드 원칙

- 언어: TypeScript 우선
- 컴포넌트: 함수형 컴포넌트
- 페이지에서 Firebase SDK 직접 호출 금지

- repository / service / function 계층 분리

---

## 1-4. 에이전트 행동 규칙

1. 모든 설명은 한국어로 작성한다.
2. 모든 주석은 한국어 우선으로 작성한다.
3. 모든 커밋 메시지는 한국어로 작성한다.
4. 임의 추정으로 스키마를 바꾸지 않는다.
5. 문서 기준 없이 기능을 추가하지 않는다.
6. 1회 수정 단위를 작게 유지한다.
7. 한 번에 여러 도메인을 동시에 건드리지 않는다.
8. 수정 전 영향 범위를 먼저 적는다.
9. 수정 후 검증 항목을 반드시 적는다.
10. 불확실하면 우회 구현보다 보수적으로 유지한다.
11. 보안 규칙을 완화하는 방향으로 문제를 해결하지 않는다.
12. 로그/감사 추적을 없애는 방향으로 리팩토링하지 않는다.
13. UI 개선 때문에 데이터 흐름을 단순화하며 무결성을 깨지 않는다.
14. EMPLOYEE 경험 최적화와 ADMIN 통제 구조를 동시에 유지한다.

---

## 1-5. 에이전트 작업 단위 규칙

1. 작업은 반드시 초원자 단위로 쪼갠다.
2. 각 작업은 15분 이내 목표로 쪼갠다.
3. 각 작업은 Explain / Commands / Verification / PASS / FAIL 구조를 가진다.
4. 코드 수정 전 대상 파일을 명시한다.
5. 새 파일 생성 시 목적과 import 위치를 명시한다.
6. 기존 기능을 제거할 때는 대체 구조를 먼저 만든다.
7. 컴파일 에러 0개를 기본 PASS 조건으로 둔다.

---

## 1-6. 구조적 금지사항

- page 컴포넌트에서 Firestore write 직접 호출 금지
- UI 컴포넌트 내부에서 role 판정 로직 중복 구현 금지
- 도메인 타입을 any로 처리 금지
- 일정 상태를 색상 문자열만으로 표현 금지
- 장비 상태를 label만 바꿔서 처리 금지
- 임시 더미 데이터가 실제 프로덕션 분기보다 우선되지 않게 할 것
- mock 데이터가 남아 있어 실제 Firebase 데이터 흐름을 가리지 않게 할 것

---

## 1-7. AGENT 작업 우선순위 원칙

1. 보안
2. 데이터 무결성
3. 상태 전이 정확성
4. 권한 분리

5. 운영 UX
6. 시각 polish

즉, 예쁜 UI보다 먼저 맞아야 하는 것은 **상태 / 권한 / 로그 / 트랜잭션**이다.

---

# 2. UI/UX 및 브랜드 1:1 치환 매핑서 (clone-spec.md)

이 장은 에이전트가 모호하게 "예쁘게 바꿔줘" 식으로 작업하지 못하도록, **직접 코드에 치환 가능한 수준의 명확한 매핑 데이터**를 정의한다.

주의: 이 프로젝트는 레스토랑/배달앱 리브랜딩 문서가 아니라 **업무관리 포털용 치환 문서**다.

---

## 2-1. 앱 식별자 (Metadata)

### 앱 이름

- 정식 표시명: HELLA company 업무관리사이트
- 보조 설명: 내부 직원 전용 운영 시스템
- 짧은 표기: HELLA OPS

### 라우팅 식별 구조

- 공통 진입: /login
- 관리자 앱: /app/*
- 사원 앱: /m/*

### 역할별 기본 진입 경로

- EMPLOYEE → /m/home
- LEADER → /m/home
- ADMIN → /app/dashboard
- SUPER_ADMIN → /app/dashboard

### 패키지 / 프로젝트 식별자 권장

- package name 예시: hella-ops-web
- firebase project alias 예시: hella-ops
- app title constant: HELLA company 업무관리사이트

---

## 2-2. 브랜드 메시지 치환 규칙

### 로그인 화면 타이틀

- 고정 텍스트: HELLA company 업무관리사이트

### 로그인 서브타이틀

- 고정 텍스트: 내부 직원 전용 운영 시스템

### 하단 보조 문구

- 고정 텍스트: 본 시스템은 내부 직원 전용입니다. 무단 접근이 제한됩니다.

### 관리자 헤더 앱명

- HELLA OPS

### 브라우저 문서 title 권장

- 로그인: HELLA company 업무관리사이트
- 관리자: HELLA OPS 관리자
- 사원: HELLA OPS 사원

---

## 2-3. 디자인 토큰 (Design Tokens)

아래 토큰은 업무용, 안정감, 가독성, 현장 조작성을 기준으로 정의한다.

### Primary

- primary-900: #0F172A
- primary-800: #1E293B
- primary-700: #334155
- primary-600: #475569
- primary-500: #64748B

### Brand Accent

- brand-main: #0F766E
- brand-main-hover: #115E59
- brand-soft: #CCFBF1
- brand-border: #99F6E4

### Background

- bg-app: #F8FAFC
- bg-surface: #FFFFFF
- bg-muted: #F1F5F9
- bg-sidebar: #0F172A

### Text

- text-primary: #0F172A
- text-secondary: #475569
- text-muted: #64748B
- text-on-dark: #FFFFFF

### Status

- status-planned: #2563EB
- status-in-progress: #16A34A
- status-completed: #475569
- status-delayed: #DC2626
- status-cancelled: #94A3B8

### Equipment Status

- equipment-available: #16A34A
- equipment-checked-out: #2563EB
- equipment-overdue: #DC2626
- equipment-disabled: #6B7280

### Alert

- alert-info-bg: #EFF6FF
- alert-info-text: #1D4ED8
- alert-warn-bg: #FFF7ED
- alert-warn-text: #C2410C
- alert-error-bg: #FEF2F2
- alert-error-text: #B91C1C
- alert-success-bg: #ECFDF5
- alert-success-text: #047857

### Border / Shadow

- border-default: #E2E8F0
- border-strong: #CBD5E1
- shadow-card: 0 1px 3px rgba(15, 23, 42, 0.08)
- shadow-popover: 0 8px 24px rgba(15, 23, 42, 0.12)

---

## 2-4. 공통 UI 치환 규칙

### 로그인 화면 배치 순서

1. 로고
2. 서비스명
3. 안내문
4. 사원 ID 입력
5. 비밀번호 입력
6. 오류 메시지
7. 로그인 버튼
8. 하단 보조 문구

### 관리자 대시보드 배치 순서

1. KPI Section
2. Alert Section
3. Quick Action Section

4. Chart Section
5. Today Schedule Section
6. Equipment Section
7. Activity Section

### 사원 홈 배치 순서

1. Header
2. Today Summary Card
3. Quick Action Grid
4. Today Schedule List
5. Equipment Status

### 사원 일정 화면 배치 순서

1. MobileHeader
2. DateScopeTabs
3. MiniCalendarSection
4. ScheduleFilterBar
5. ScheduleListSection
6. ScheduleDetailBottomSheet

### 사원 마감 화면 배치 순서

1. MobileHeader
2. ScheduleSummaryCard
3. WorkStatusStepper
4. StartSection
5. EndSection
6. NotesSection
7. BeforePhotoSection
8. AfterPhotoSection
9. ValidationAlertArea
10. BottomActionBar

### 관리자 장비관리 화면 배치 순서

1. PageHeader
2. ActionBar
3. FilterPanel
4. EquipmentTable
5. EquipmentDetailDrawer
6. AlertPanel

---

## 2-5. 메뉴 배열 및 상수 데이터

### 관리자 사이드바 메뉴 배열

```
[
 '대시보드',
```

```
 '사업분야별 관리메뉴',
 '업무메뉴별 메뉴얼',
 '정기청소 관리',
 '회사 일정관리',
 '스케줄별 일정표',
 '스케줄 마감',
 '사원관리',
 'NFC 장비 반출·반입관리',
 '설정'
]
```

### 사원 하단 탭 배열

```
[
 '홈',
 '일정',
 '장비',
 '메뉴얼',
 '내정보'
]
```

### 사원 홈 Quick Action 배열

```
[
 '작업 시작',
 '작업 종료',
 '장비 반출',
 '장비 반입'
]
```

### 관리자 Quick Action 배열

```
[
 '일정 생성',
 '사원 등록',
 '장비 등록',
 '메뉴얼 등록'
]
```

### 일정 상태 라벨 배열

```
['PLANNED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED', 'DELAYED']
```

### 사원 작업 상태 라벨 배열

```
['BEFORE_START', 'IN_PROGRESS', 'END_RECORDED', 'CLOSING_SUBMITTED', 'COMPLETED']
```

### 장비 상태 라벨 배열

```
['AVAILABLE', 'CHECKED_OUT', 'OVERDUE', 'DISABLED']
```

### 사진 타입 배열

```
['BEFORE', 'AFTER']
```

---

## 2-6. 텍스트 치환 규칙

### 로그인 필드 라벨

- employeeId label: 사원 ID
- password label: 비밀번호

### 로그인 버튼

- 기본: 로그인
- 로딩: 로그인 중...

### 사원 일정 상태 뱃지

- PLANNED → 예정
- IN_PROGRESS → 진행중
- COMPLETED → 완료
- DELAYED → 마감 필요 또는 지연
- CANCELLED → 취소

### 장비 상태 뱃지

- AVAILABLE → 보유
- CHECKED_OUT → 반출중
- OVERDUE → 미반입
- DISABLED → 비활성

### 에러 문구 공통

- 데이터를 불러오지 못했습니다.
- 다시 시도해 주세요.
- 권한이 없습니다.
- 현재 사용이 중지된 계정입니다.
- 등록되지 않은 장비 태그입니다.

---

## 2-7. UI 간격 및 크기 기준

### 공통 카드

- border-radius: 16px
- padding: 16px ~ 20px
- shadow: 약한 카드 그림자

### 모바일 주요 버튼

- 최소 높이: 52px
- 최소 터치 영역: 44px 이상

### 입력 필드

- 높이: 48px ~ 52px
- 라벨과 필드 간격: 6px ~ 8px
- 필드 간 간격: 12px ~ 16px

### 목록 카드 타이포

- 제목: 16~18px / bold
- 보조정보: 13~14px
- 상태/시간: 12~14px

---

# 3. 초원자 단위 실행 계획서 (SKILL.md Workflow)

이 장은 에이전트가 실제 구현할 때 따라야 하는 **작업 단위 쪼개기 기준**이다.

원칙: - 한 스텝은 15분 이내 목표 - 한 스텝은 한 화면 또는 한 로직 단위만 다룬다 - 한 스텝은 명확한 PASS/FAIL 기준이 있어야 한다 - FAIL 시 자가 치유 방향이 정의되어 있어야 한다

---

## 3-1. Step 01 — 프로젝트 골격 생성

### Explain

React + Vite + TypeScript + Tailwind + Firebase 기준으로 프로젝트 기본 골격을 만든다.

### Commands

- Vite React TS 프로젝트 생성
- Tailwind 설정
- 기본 src/app, src/pages, src/features, src/services, src/repositories, src/types, src/routes 구조 생성
- firebase 초기 설정 파일 추가

### Verification

- `npm run build` 에러 0개

- 기본 App 렌더링 성공
- Tailwind 클래스 적용 확인

### PASS

- 프로젝트가 정상 빌드되고 기본 화면이 열린다.

### FAIL 및 대처

- Tailwind 미적용 시 content 경로와 CSS import 다시 점검
- TS path alias 오류 시 tsconfig / vite alias 정렬 후 재검증

---

## 3-2. Step 02 — 라우팅 및 권한 분기 골격

### Explain

로그인 후 역할별로 `/app/*` 또는 `/m/*` 로 분기되는 기본 라우팅 뼈대를 만든다.

### Commands

- React Router 설치 및 routePaths 정의
- `/login`, `/app/dashboard`, `/m/home` 기본 페이지 생성
- RequireAuth, RequireAdmin, RequireActiveUser 가드 생성

### Verification

- 비로그인 상태에서 보호 라우트 접근 시 `/login` 이동
- 로그인 mock 상태에서 role별 라우트 분기 확인

### PASS

- EMPLOYEE는 `/m/home`, ADMIN은 `/app/dashboard` 로 이동한다.

### FAIL 및 대처

- 무한 리다이렉트 발생 시 auth bootstrap과 guard 조건 분리
- role 분기 오류 시 postLoginRoute resolver 재검증

---

## 3-3. Step 03 — 로그인 화면 구현

### Explain

로그인 와이어프레임 기준으로 실사용 가능한 로그인 화면을 구현한다.

### Commands

- LoginPage 생성
- employeeId / password / error / loading 상태 구현
- BrandHeader, LoginForm, FooterInfoBlock 분리

### Verification

- 빈 값 제출 시 필드 에러 표시
- submitting 중 버튼/입력 비활성
- 모바일 키보드 노출 시 레이아웃 깨짐 없음

### PASS

- 로그인 화면 구조가 문서 정의 순서대로 렌더링된다.

### FAIL 및 대처

- input/버튼 겹침 시 mobile spacing 재조정
- 에러 메시지 위치 불안정 시 ErrorAlert 영역 고정

---

## 3-4. Step 04 — Firebase Auth + users 프로필 조회 연결

### Explain

로그인 화면을 실제 Auth/Firestore profile 조회와 연결한다.

### Commands

- authService.signInWithEmployeeCredentials 구현
- usersRepository.fetchUserProfile 구현
- resolvePostLoginRoute 구현

### Verification

- 잘못된 자격 증명 시 일반화된 오류 표시
- 비활성 계정 차단
- role 누락 계정 차단

### PASS

- 정상 로그인 시 역할별 홈으로 이동한다.

### FAIL 및 대처

- Auth 성공 후 profile 미조회 시 uid 기반 조회 로직 점검
- 비활성 계정이 통과하면 profile status 검사 추가

---

## 3-5. Step 05 — 공통 레이아웃 생성

### Explain

관리자/사원 레이아웃 뼈대를 나눠 만든다.

### Commands

- AdminLayout 생성
- EmployeeLayout 생성
- Sidebar / Header / BottomTab 기본 구조 생성

### Verification

- `/app/*` 에서 관리자 셸 렌더링
- `/m/*` 에서 사원 셸 렌더링

### PASS

- 레이아웃이 역할별로 분리된다.

### FAIL 및 대처

- 공통 CSS 충돌 시 layout 범위 클래스로 격리

---

## 3-6. Step 06 — 관리자 대시보드 1차 구현

### Explain

KPI, Alert, Quick Action 중심의 관리자 첫 화면을 구현한다.

### Commands

- KPISection 생성
- AlertSection 생성
- QuickActionSection 생성
- 더미 데이터 기반 초기 렌더링

### Verification

- KPI 4개 카드 노출
- 미마감/미반입 alert 노출
- Quick Action 버튼 동작 확인

### PASS

- 관리자 대시보드 1차 화면이 문서 구조대로 보인다.

### FAIL 및 대처

- 위젯 간 레이아웃 깨짐 시 12-column grid 조정

---

## 3-7. Step 07 — 사원 홈 1차 구현

### Explain

사원 홈의 업무 시작 허브 화면을 구현한다.

### Commands

- Header
- Today Summary Card
- Quick Action Grid
- Today Schedule List
- Equipment Status 구성

### Verification

- 모바일에서 세로 스크롤 정상
- 버튼 4개 노출
- 오늘 일정 카드 렌더링

### PASS

- 사원 홈이 업무 시작 허브 역할로 보인다.

### FAIL 및 대처

- 버튼이 과밀하면 2x2 grid 재조정

---

## 3-8. Step 08 — 관리자 일정관리 목록/캘린더 화면

### Explain

관리자가 일정 목록과 캘린더를 동시에 볼 수 있는 화면을 구현한다.

### Commands

- AdminScheduleListPage 생성
- FilterPanel 생성
- CalendarPanel 생성
- ScheduleListPanel 생성

### Verification

- 날짜 클릭 시 리스트 필터 반영
- 상태/카테고리/사원 필터 반영

### PASS

- 일정 목록과 캘린더가 동작한다.

### FAIL 및 대처

- 필터 변경마다 과한 재요청이면 debounce 적용

---

## 3-9. Step 09 — 일정 생성/수정 폼 구현

### Explain

일정 생성과 수정에 필요한 필드 및 검증 UI를 구현한다.

### Commands

- AdminScheduleFormPage 생성
- FormSectionBasic / Site / Workers / Notes 생성
- ValidationSummary 구현

### Verification

- 제목/날짜/시작시간 필수 검증
- 종료예정시간 역전 차단
- 비활성 사원 배정 목록 제외

### PASS

- 생성 폼이 저장 가능한 상태까지 동작한다.

### FAIL 및 대처

- worker selector 오류 시 users query 조건 재검토

---

## 3-10. Step 10 — 일정 상세 화면 구현

### Explain

일정 상세, 배정 사원, 마감 정보, 사진 정보를 볼 수 있게 한다.

### Commands

- AdminScheduleDetailPage 생성
- SummaryCard / TabNavigation / TabAssignedWorkers / TabClosingInfo / TabPhotos 구현

### Verification

- 탭 전환 정상
- 배정 사원 상태 표시 정상
- 마감/사진 탭 데이터 자리 정상

### PASS

- 관리자 상세 화면에서 일정 운영 정보가 보인다.

### FAIL 및 대처

- 탭 간 데이터 꼬임 시 query key 분리

---

## 3-11. Step 11 — 사원 일정 화면 구현

### Explain

사원이 본인 일정만 보고 작업 시작/마감 화면으로 이동할 수 있게 한다.

### Commands

- EmployeeSchedulePage 생성
- DateScopeTabs / MiniCalendar / ScheduleList / DetailBottomSheet 구현

### Verification

- 오늘/이번주/전체 탭 동작
- 카드 클릭 시 상세 바텀시트 오픈
- 상세에서 시작/마감 CTA 노출 규칙 확인

### PASS

- 사원 일정 화면이 본인 일정 중심으로 동작한다.

### FAIL 및 대처

- 일정 과다 렌더링 시 virtualization 또는 pagination 검토

---

## 3-12. Step 12 — 사원 시작/종료/마감 화면 UI 구현

### Explain

현장 실행용 핵심 화면을 단계형 구조로 구현한다.

### Commands

- EmployeeClosingPage 생성
- StartSection / EndSection / NotesSection / BeforePhotoSection / AfterPhotoSection / BottomActionBar 구현

### Verification

- 시작 없이 종료 비활성
- 사진 업로드 UI 표시

- ValidationAlertArea 동작

### PASS

- 사용자가 시작→종료→사진→마감 흐름을 이해할 수 있다.

### FAIL 및 대처

- 액션 바가 모바일 키보드에 가리면 sticky 정책 조정

---

## 3-13. Step 13 — 장비관리 관리자 화면 구현

### Explain

장비 목록/상세/필터/미반입 강조를 구현한다.

### Commands

- AdminEquipmentPage 생성
- EquipmentTable / EquipmentDetailDrawer / AlertPanel 구현

### Verification

- 상태별 색상 노출
- 상세 drawer 오픈
- 미반입 장비 강조 표시

### PASS

- 관리자 장비 흐름 추적 화면이 보인다.

### FAIL 및 대처

- table 과밀 시 tablet 이하 card list fallback 추가

---

## 3-14. Step 14 — 사원 반출/반입 화면 구현

### Explain

사원이 NFC 반출/반입 모드에서 장비를 처리할 수 있는 화면을 구현한다.

### Commands

- EmployeeEquipmentOutPage 생성
- EmployeeEquipmentInPage 생성
- mode banner / scan button / result card / confirm action 구현

### Verification

- 모드별 라벨이 명확히 다름

- 스캔 전/후 상태 구분 가능

### PASS

- 반출/반입 흐름이 헷갈리지 않는다.

### FAIL 및 대처

- mode 혼선 시 화면 상단에 명시적 모드 배지 추가

---

## 3-15. Step 15 — Firestore Rules 1차 적용

### Explain

기본 컬렉션 접근 제어와 본인/관리자 분리를 적용한다.

### Commands

- users, schedules, schedule_workers, schedule_closings, schedule_photos, equipments, equipment_logs, audit_logs, settings rules 작성
- Emulator 테스트 추가

### Verification

- 비로그인 접근 차단
- EMPLOYEE 타인 데이터 접근 차단
- equipment_logs 직접 쓰기 차단

### PASS

- 핵심 보안 구멍이 1차 봉쇄된다.

### FAIL 및 대처

- 테스트 실패 시 필드 whitelist와 role helper 분리 재검토

---

## 3-16. Step 16 — Functions: 일정 생성 + 배정

### Explain

일정 생성 시 schedule와 schedule_workers를 한 번에 생성한다.

### Commands

- createScheduleWithWorkers 함수 작성
- 입력 검증
- 트랜잭션 생성
- audit log 기록

### Verification

- schedule 생성 후 worker 문서 수 일치
- 비활성 사원 배정 차단

### PASS

- 관리자 일정 생성이 원자 처리된다.

### FAIL 및 대처

- worker 생성 누락 시 batch/transaction 범위 점검

---

## 3-17. Step 17 — Functions: 작업 시작/종료/마감

### Explain

사원 작업 상태를 서버 함수로만 확정한다.

### Commands

- recordWorkStart
- recordWorkEnd
- submitClosing
- recomputeScheduleStatus 작성

### Verification

- 시작 전 종료 차단
- 종료 > 시작 검증
- 마감 제출 후 상태 동기화

### PASS

- 클라이언트 우회 없이 상태가 서버에서 정해진다.

### FAIL 및 대처

- 상태 역전 발생 시 prev/next 전이 검증 강화

---

## 3-18. Step 18 — Functions: NFC 반출/반입 트랜잭션

### Explain

장비 상태 변경과 로그 생성을 한 번에 처리한다.

### Commands

- processNfcScan

- checkoutEquipment
- checkinEquipment
- forceUpdateEquipmentStatus 구현

### Verification

- AVAILABLE만 반출 가능
- CHECKED_OUT/OVERDUE만 반입 가능
- 로그 동시 생성

### PASS

- 장비 상태와 로그가 항상 일치한다.

### FAIL 및 대처

- 상태/로그 불일치 시 한 트랜잭션으로 재구성

---

## 3-19. Step 19 — Storage Rules + 사진 업로드 연동

### Explain

사진 메타와 실제 파일 권한을 일치시킨다.

### Commands

- storage.rules 작성
- uploadSchedulePhoto 구현
- schedule_photos metadata 생성

### Verification

- 업로더 본인/관리자만 접근
- 업로드 실패 시 retry 가능

### PASS

- 사진 업로드가 보안과 UX를 함께 만족한다.

### FAIL 및 대처

- Firestore와 Storage 소유권 조건 불일치 시 경로 규칙 재정의

---

## 3-20. Step 20 — 운영 안정화 테스트

### Explain

핵심 시나리오를 end-to-end 기준으로 검증한다.

### Commands

- 로그인 → 일정 생성 → 사원 시작 → 종료 → 사진 → 마감 → 관리자 검토 → 장비 반출/반입 시나리오 점검
- Rules/Functions/emulator 테스트 정리

### Verification

- 컴파일 에러 0개
- 권한 우회 0개
- 상태 전이 오류 0개
- 로그 누락 0건

### PASS

- 운영 시나리오가 끊기지 않는다.

### FAIL 및 대처

- 어느 단계에서든 상태 동기화 누락 시 해당 함수 책임 범위 재설계

---

# 4. 백엔드 연동 및 예외 상태 정의서

이 장은 UI가 아니라 실제 데이터가 어떻게 들어오고, 없을 때 어떻게 처리하고, 실패하면 무엇을 보여줄지 정의한다.

핵심 원칙: - 화면은 항상 실제 데이터 모델 기준으로 그린다. - 값이 없을 때 에러를 터뜨리기보다 우아하게 숨기거나 대체 문구를 쓴다. - 단, 보안/권한/상태 위반은 명확히 차단한다.

---

## 4-1. 데이터 바인딩 규칙 — 로그인

### 화면 데이터

- 로고: settings.loginLogoUrl 또는 기본 로고
- 서비스명: settings.companyName 또는 고정값 HELLA company 업무관리사이트
- employeeId input: form.employeeId
- password input: form.password
- 오류 메시지: authErrorState

### 예외 처리

- 로고 없음 → 텍스트 로고 fallback
- companyName 없음 → HELLA company 업무관리사이트 고정 사용
- users profile 없음 → 접근 차단 + 일반 오류 문구

---

## 4-2. 데이터 바인딩 규칙 — 관리자 대시보드

### KPI 카드

- 오늘 일정 수 ← schedules where date=today count
- 진행중 일정 수 ← schedules.status == IN_PROGRESS count
- 미마감 건수 ← schedule_workers / schedule_closings 미완료 조합 계산
- 미반입 장비 수 ← equipments.status in [CHECKED_OUT, OVERDUE] count

### 알림 리스트

- 미마감 Top N ← needsClosing query
- 미반입 장비 Top N ← outstandingEquipments query

### 빠른 액션

- 고정 route config 기반

### 예외 처리

- KPI 데이터 없음 → 0 표시
- alert 목록 0건 → "현재 즉시 처리할 항목이 없습니다" 표시
- 차트 데이터 없음 → empty chart placeholder
- 일부 위젯 실패 → 위젯 단위 error card

---

## 4-3. 데이터 바인딩 규칙 — 사원 홈

### Today Summary

- 오늘 일정 수 ← getTodaySchedules(userId).length
- 진행중 일정 ← myWorkStatus == IN_PROGRESS count
- 완료 일정 ← myWorkStatus == COMPLETED or closing submitted count

### Today Schedule List

- 본인 배정 schedule_cards

### Equipment Status

- 내 현재 장비 ← equipments.currentHolderUserId == uid

### 예외 처리

- 오늘 일정 0건 → "오늘 일정 없음" 카드
- 장비 0건 → "현재 보유 장비 없음" 표시
- 네트워크 지연 → 스켈레톤 후 재시도 버튼

---

## 4-4. 데이터 바인딩 규칙 — 관리자 일정관리

### 목록

- schedules 컬렉션 + filter 조합

### 캘린더 요약

- dateKey 기반 summary map

### 상세

- schedules/{id}
- schedule_workers by scheduleId
- schedule_closings by scheduleId
- schedule_photos by scheduleId

### 예외 처리

- 일정 0건 → empty list + empty calendar state 유지
- filter 결과 0건 → “조건에 맞는 일정이 없습니다”
- 삭제 대신 취소 상태 사용
- 이미 시작된 일정 배정 해제 시 경고 모달

---

## 4-5. 데이터 바인딩 규칙 — 사원 일정 화면

### 리스트 데이터

- schedule_workers where workerId = uid
- schedules 표시용 정보 조합

### 상세 바텀시트

- title ← schedules.scheduleTitle
- address ← schedules.siteAddress
- startTime ← schedules.startTime
- expectedEndTime ← schedules.expectedEndTime
- status ← schedules.status + myWorkStatus 조합

### 예외 처리

- 본인 배정 아닌 scheduleId 접근 → 차단 메시지 + 리스트 유지
- 특정 날짜 일정 0건 → EmptyStateSection
- 취소 일정 표시 정책은 muted style 또는 숨김 정책 선택

---

## 4-6. 데이터 바인딩 규칙 — 사원 시작/종료/마감 화면

### 일정 요약 카드

- title ← schedules.scheduleTitle
- category/subcategory ← categories join or cached names
- date/time ← schedules.scheduleDate/startTime
- address ← schedules.siteAddress
- current status ← schedule_workers.workStatus + closing 상태 조합

### 시작/종료 입력

- startAt ← schedule_workers.actualStartAt
- endAt ← schedule_workers.actualEndAt

### 특이사항

- notes ← local state + draft 저장값

### 사진

- beforePhotos ← schedule_photos where type=BEFORE
- afterPhotos ← schedule_photos where type=AFTER

### 예외 처리

- schedule 조회 실패 → "현장 정보를 불러오지 못했습니다" + 재시도
- 시작 없이 종료 시도 → blocking error
- 종료 < 시작 → blocking error
- 업로드 실패 사진 존재 → 마감 제출 차단
- 이미 제출 완료 → 읽기 전용 전환

---

## 4-7. 데이터 바인딩 규칙 — 관리자 장비관리

### 목록

- equipments 전체 또는 필터 조회

### 상세 Drawer

- equipment 기본 정보 ← equipments/{id}
- 로그 ← equipment_logs where equipmentId
- 현재 사용자 ← equipments.currentHolderName

### 미반입 리스트

- equipments.status == OVERDUE or CHECKED_OUT + overdue 계산

### 예외 처리

- 장비 0건 → "등록된 장비가 없습니다"

- 로그 0건 → “이력이 없습니다”
- 중복 ACTIVE 매핑 감지 시 관리자 오류 배너

---

## 4-8. 데이터 바인딩 규칙 — 사원 반출/반입

### 반출 화면

- mode = CHECK_OUT
- 스캔 결과 ← processNfcScan
- 장비 정보 ← equipmentName/currentStatus/allowedAction/message
- 사유 ← form.reason
- 위치 ← form.useLocation
- 메모 ← form.note

### 반입 화면

- mode = CHECK_IN
- 스캔 결과 ← processNfcScan
- 반출자 / 반출시각 표시 ← equipments.currentHolderName / lastCheckoutAt
- 메모 ← form.note

### 예외 처리

- Web NFC 미지원 → 명확한 안내 + 수동 선택 정책 여부 표시
- 태그 미등록 → 사원 차단 / 관리자 등록 유도
- 이미 반출중 장비 반출 시도 → 현재 보유자/시간 표시
- 보관 상태 장비 반입 시도 → 차단 메시지
- 서버 지연 → 로딩 유지 + 중복 클릭 차단

---

## 4-9. 초기 더미(Mock) 데이터 처리 원칙

### 원칙

- 기능 개발 초기에는 mock 허용
- 그러나 실제 Firebase 연결 전환 시 mock이 우선되면 안 됨
- mock은 `src/mocks/*` 또는 `__fixtures__/*` 로 분리
- page에서 mock import 직접 사용 금지
- repository 계층에서 only-dev 조건으로 주입

### 컬렉션별 기본 원칙

- schedules 초기값: []
- equipments 초기값: []
- manuals 초기값: []
- users 초기값: []

### 예외

디자인 검증용 샘플은 허용하되, 주석으로 명시한다. - `개발용 샘플 데이터` - `실제 Firebase 연결 시 제거 또는 dev-only 조건으로 제한`

---

## 4-10. Boring States 정의

이 프로젝트에서 boring state는 "에러는 아니지만 반드시 고려해야 하는 빈 상태 / 느린 상태 / 애매한 상태 / 부분 실패 상태"를 뜻한다.

### 공통 boring states

1. 데이터 0건
2. 부분 로딩
3. 느린 네트워크
4. 권한은 있으나 표시할 데이터 없음
5. 일부 위젯만 실패
6. 업로드 일부 실패
7. 상태는 유효하지만 사용자가 다음 액션을 모를 때

---

## 4-11. Boring States — 화면별 정책

### 로그인

- 로고 없음 → 텍스트 로고 fallback
- 네트워크 느림 → 버튼 loading 유지
- 인증 실패 → 일반화된 오류 문구
- 프로필 누락 → 접근 차단

### 관리자 대시보드

- KPI 0건 → 0 표시
- 오늘 일정 없음 → empty widget
- alert 0건 → 정상 안내문
- 일부 위젯 실패 → 해당 위젯만 에러 카드

### 사원 홈

- 오늘 일정 없음 → 업무 없음 카드
- 보유 장비 없음 → 장비 없음 표시
- 최근 작업 없음 → 최근 기록 없음 표시

### 관리자 일정관리

- 필터 결과 0건 → 빈 리스트 문구 + 필터 유지
- worker 0명 배정 → 저장 경고는 띄우되 정책상 저장 허용 가능
- category/subcategory 미로드 → selector disabled + retry

### 사원 일정

- 특정 날짜 일정 없음 → EmptyState 유지
- 전체 탭 0건 → “배정된 일정이 없습니다”
- 상세 조회 실패 → 바텀시트 내 재시도

### 사원 마감

- 비포 0장 → warning 또는 blocking 정책 선택
- 애프터 0장 → warning 또는 blocking 정책 선택
- notes 빈 값 → 허용
- 종료 미입력 → blocking
- 업로드 실패 남음 → blocking
- 제출 후 재진입 → 읽기 전용

### 장비 반출/반입

- NFC 미지원 → graceful fallback 안내
- 태그 미등록 → 처리 차단
- 충돌 발생 → 재조회 유도
- 스캔은 됐지만 allowedAction=NONE → 이유 메시지 노출

### 관리자 장비관리

- 장비 없음 → empty table
- 로그 없음 → empty log panel
- overdue 없음 → 정상 상태 안내

---

## 4-12. Graceful Fallback 원칙

### 숨겨야 하는 경우

- 권한 없는 섹션
- 데이터 0건인데 섹션 존재 의미가 약한 경우

### 남겨야 하는 경우

- 사용자가 다음 행동을 알아야 하는 경우
- empty state 자체가 중요한 안내인 경우
- 일정/장비/로그처럼 운영 핵심인 경우

### 예시

- 추천 메뉴 같은 섹션이 없으므로 숨김보다 명시적 empty state가 적절하다.
- 오늘 일정이 0건이면 섹션을 숨기지 말고 “오늘 일정 없음”을 보여준다.
- 장비 로그 0건도 이력이 없음을 보여주는 것이 낫다.

---

## 4-13. 백엔드 연동 금지 원칙

- 클라이언트 timestamp를 진실로 쓰지 않는다.
- 상태 판정을 클라이언트에서 끝내지 않는다.
- processNfcScan 결과만 믿고 상태 변경하지 않는다.
- Firestore read 결과를 수정 없이 재사용할 때도 role/status 재검증을 누락하지 않는다.

---

## 4-14. 최종 결론

이 문서는 에이전트가 반드시 알아야 할 4개의 실전 문서를 하나로 합친 것이다.

핵심은 다음이다.

1. AGENTS.md: 절대 망가뜨리면 안 되는 규칙
2. clone-spec.md: 모호하지 않은 1:1 UI/브랜드 치환 데이터
3. SKILL.md Workflow: 15분 단위 초원자 실행 계획
4. Backend States: 실제 데이터 연결과 boring states 대처 기준

이 4개가 준비되어야, 에이전트가 이 프로젝트를 함부로 건드리지 않고 **안전하게, 작은 단위로, 검증 가능하게** 구현할 수 있다.